

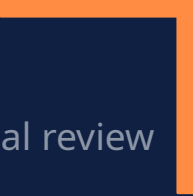


HERMES / ARCHITECTURE BRIEF

Memory System Decisions

*Three systems, one stack — what we kept, what we cut,
and why the embedder finally got fixed.*

May 2026



Prepared for technical review

■ Three things were being conflated

Operator confusion was warranted — we had three memory ideas in play with similar names, and the team wasn't sure which one was actually doing the work.

SYSTEM 1

MemOS v1 server

IDLE

Standalone HTTP service on :8001. Qdrant + Neo4j + SQLite. Cube-based isolation enforced server-side via API-key → user_id binding. The original production target.

SYSTEM 2

v2 memos plugin

ACTIVE

Hermes-native plugin. Single SQLite + UI on :18800. Reward backprop, L1/L2/L3 layers, automatic skill crystallization. Isolation by row-level namespace tuple.

SYSTEM 3

The hub

PLANNED

Future cross-machine sync feature. Config schema and a share_scope='hub' value exist; zero code reads them. Failing systemd unit was trying to start it.

■ What's actually running

COMPONENT	ROLE	STATUS	LOCATION
v2 memos-local-plugin	Sole active memory provider	ACTIVE	~/.hermes/memos-plugin/ :18800
MemOS v1 server	HTTP API; ceo-cube empty (zero writes)	STOPPED	localhost:8001 (disabled today)
memos-toolset	v1 client; plugin.yaml.disabled-2026-05-12	INERT	~/.hermes/plugins/memos-toolset/
memos-hub.service	Tries to start non-existent hub mode	DISABLED	scripts/migration/bootstrap-hub.sh
Hermes agent gateways	Per-profile processes (each writes to v2)	ACTIVE	hermes-gateway-{sergio,hr-agent,...}

→ One system is actually doing the work. Two are inert. One was thrashing.

■ The cross-contamination worry — and why it was moot

THE CONCERN

Why we thought agents were sharing memory

- Shared ~/.hermes/.env across every agent
- One MEMOS_USER_ID=ceo, one MEMOS_CUBE_ID=ceo-cube
- One API key — server sees one identity for every agent
- Server's cube-ownership check can't distinguish callers
- **Therefore: every write lands in the same cube**

THE REALITY

v1 was effectively unused all along

- memos-toolset (v1 client) was disabled on 2026-05-12
- Profiles already switched to provider: memtensor (v2)
- Probed v1 cube directly: text_mem=[], total_nodes=0
- v2 uses a different isolation model (next slide)
- **Net effect: zero actual contamination, ever**

■ How v2 isolation actually works

Not cubes. Row-level multi-tenancy with a namespace tuple stamped on every row.

```
core/runtime/namespace.ts
```

```
// every read is filtered through this
function isVisibleTo(row, caller) {
  if (row.share_scope !== 'private') return true;
  return (
    row.owner_agent_kind === caller.agentKind &&
    row.owner_profile_id === caller.profileId
  );
}
```

```
// every row carries these columns
owner_agent_kind  TEXT NOT NULL
owner_profile_id  TEXT NOT NULL
share_scope       TEXT DEFAULT 'private'
```

THE PRIMITIVE

(agentKind, profileId, workspaceId)

Plus a share_scope column on every row. Default is private. The plugin derives profileId from \$HERMES_HOME — so each agent process gets its own namespace automatically as long as it launches with its own profile dir.

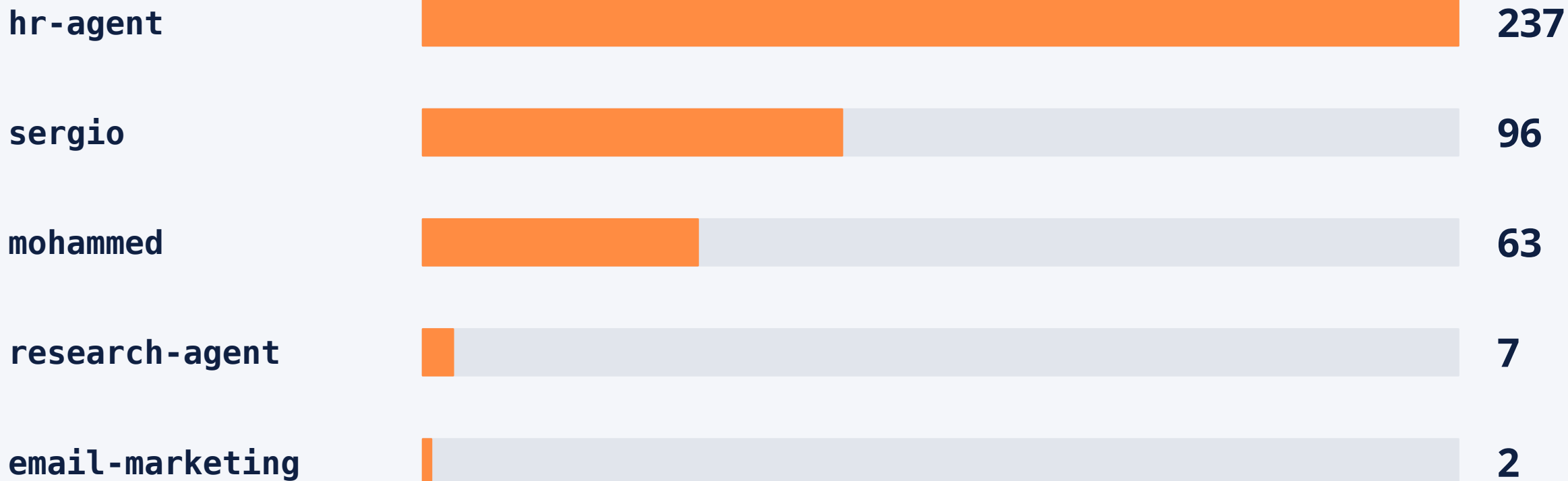
WHY IT WORKS

One SQLite, many tenants — no cubes, no per-agent files
Enforced at every retrieval, not by trust

No server, no API, no credential management

■ Isolation is working — verified

```
SELECT owner_profile_id, COUNT(*) FROM traces GROUP BY owner_profile_id
```



✓ No "default" bucket — every agent's writes land under its own profileId. The earlier worry was real for v1 but not for v2.

■ share_scope: design intent vs reality today

SCOPE	DESIGN INTENT	TODAY'S BEHAVIOR	AS DESIGNED
private	Visible only to the writing profile	Only one that actually filters	✓
local	Visible across profiles on this machine	Visible to all (same as public)	✗
public	Visible to everyone, everywhere	Visible to all	✓
hub	Visible across machines via hub sync	Visible to all — hub never wired	✗

Implementation collapses three of four scopes into one. Tag rows with intended scope anyway — when the hub ships, the data already knows where it should go.

■ v1 vs v2 — what each is actually good at

MemOS v1 server

Better long-term backbone for production

- Server-enforced cube isolation via API-key → user_id
- Multi-machine ready — HTTP service, deploy anywhere
- CompositeCubeView: first-class CEO cross-agent read
- Mature storage stack (Qdrant + Neo4j + SQLite)
- Decoupled from Hermes — wires to any agent runtime

v2 memos-local-plugin

Better single-machine research + DX

- Built-in UI: memory browser, episode explorer, skill library
- Reward backprop + automatic skill crystallization
- L1/L2/L3 memory layers (working/episodic/semantic)
- Hooks deep into Hermes (on_delegation, post_tool_call)
- Zero infra: single SQLite, no Qdrant/Neo4j to babysit

VERDICT

■ Decision: stay on v2 for now

VERDICT

Single-tier: v2 plugin only

Revisit v1 when multi-machine deployment becomes a real need, or when an external agent runtime needs to hit the same memory.

01

Single-developer research rig

We're optimizing for iteration speed, not multi-tenant production.

02

The UI matters

Memory browser + skill library make the system actually inspectable.

03

Auto-skill crystallization

Reward backprop → L2 promotion → YAML skill is genuinely novel and only in v2.

04

Isolation already works

Per-profile row tagging is enforced, verified, sufficient.



■ Model assignments by task

Different parts of the memory pipeline want different model temperaments. We split the slots accordingly.

Extraction / summarization

Per-turn capture, summary chunking, injection



DeepSeek V3

deepseek-chat

Non-thinking model. MiniMax pollutes extraction output with <think> tags that break the parser. Constrained generation — bigger isn't better.

Skill crystallisation

L2 → active skill induction, reflection synthesis



MiniMax M2.7

MiniMax-M2.7 · Anthropic-compatible endpoint

Synthesis-heavy. Reasoning helps. Output post-processed into YAML — <think> tags get stripped cleanly. Lower call frequency means cost matters less.

Embedding

Trace summary + action vectors, semantic search



BGE-large-en-v1.5

Xenova/bge-large-en-v1.5 (local, 1024d)

Beat both MiniLM and gemini-embedding-2 on our actual data. Local — no API, no rate limits, no privacy footprint. See benchmark, next slides.

■ Embedder benchmark: how we tested

01

Pull real traces

200 trace summaries from the live v2 DB (read-only). Mixed profiles, real content — embedder choice, file transfer, idealista, plugin architecture.

02

Embed with all three

MiniLM-L6-v2 (current), BGE-large-en-v1.5 (proposed local), gemini-embedding-2 (proposed cloud). BGE used recommended query prefix.

03

Query set + retrieval

8 realistic memory queries. For each: top-5 retrieval from every model. Cosine on L2-normalized vectors.

04

LLM-as-judge

DeepSeek V3 rated each (query, snippet) pair on 0/1/2 relevance. Judged the UNION of top-5 across all models — no free passes.

METRICS: Precision@5 • MRR • nDCG@5 • pairwise top-5 overlap (Jaccard) • top-1 cosine similarity

RESULTS

■ 3-way benchmark: BGE-large wins everything

MODEL	DIM	P@5	MRR	nDCG@5	RUNTIME	COST	VERDICT
MiniLM-L6-v2	384	0.450	0.688	0.628	Local (Xenova)	Free	<i>baseline</i>
BGE-large-en-v1.5	1024	0.500	0.781	0.725	Local (Xenova)	Free	WINNER
gemini-embedding-2	3072	0.425	0.750	0.662	Google API	Free tier / \$0.20·M	

Δ vs MiniLM

+11% P@5 · +14% MRR · +15% nDCG@5

Δ vs Gemini-2

+18% P@5 · +4% MRR · +10% nDCG@5



■ Why BGE-large is the pick

Q U A L I T Y

Top of every quality metric

P@5, MRR, nDCG@5 — all three. Q4 (Idealista cluster) and Q7 (skill crystallization) saw it find content the others missed entirely.

L O C A L

No API dependency

Runs in the Hermes process via Xenova/Transformers.js ONNX. No network round-trips, no rate limits, no Google account to manage.

P R I V A C Y

Memory never leaves the box

Every other option meant agent memory content was sent to a third-party API. BGE-large keeps it on the machine where it belongs.

F O O T P R I N T

Acceptable resource cost

~1.5 GB RAM working set; machine has ~9 GB free. ~150-300ms per embedding on CPU. The hook is async — doesn't block the agent reply.

■ Cleanups: what we removed

Fewer moving parts. Each removal is reversible — none of these are destructive deletes, just service stops and config swaps.

memos-hub.service

DISABLED

Was thrashing in an auto-restart loop, calling bootstrap-hub.sh which tries to launch a hub mode the plugin doesn't implement. Zero functional impact.

memos-server.service (v1)

STOPPED

Healthy but unused — cube was empty, memos-toolset client already disabled. Frees Qdrant + Neo4j + Python memory. Restart anytime if we revisit v1.

gemini-embedding-2 config

REPLACED

Briefly swapped in during the embedder eval. Beaten by BGE-large on every quality metric. Reverted to local embedder before any data was written.

■ Where we landed: config.yaml

```
~/.hermes/memos-plugin/config.yaml

embedding:
  provider: local
  model: Xenova/bge-large-en-v1.5      # +11-18% over MiniLM / Gemini-2
  dimensions: 1024

llm:
  provider: openai_compatible
  endpoint: https://api.deepseek.com
  model: deepseek-chat                # extraction / summarisation

skillEvolver:
  provider: anthropic
  endpoint: https://api.minimax.io/anthropic
  model: MiniMax-M2.7                 # crystallisation / L2 induction

hub:
  enabled: true                       # cosmetic – feature unimplemented upstream
```

Isolation is per-profile via \$HERMES_HOME → profileId. No cubes, no shared credentials. Verified on the running DB.

Migration steps remaining

A clean one-minute downtime; no recoverable data is lost (the 30 skills were all status=candidate).

01

Stop Hermes agents

```
systemctl --user stop hermes-gateway.service hermes-gateway-*
```

02

Wipe v2 DB (384d → 1024d incompatible)

```
rm -f ~/.hermes/memos-plugin/data/memos.db*
```

03

Restart agents — DB and embeddings rebuild via BGE

```
systemctl --user start hermes-gateway.service hermes-gateway-*
```

04

Tail one agent — verify BGE model loads cleanly

```
journalctl --user -u hermes-gateway-sergio -f
```